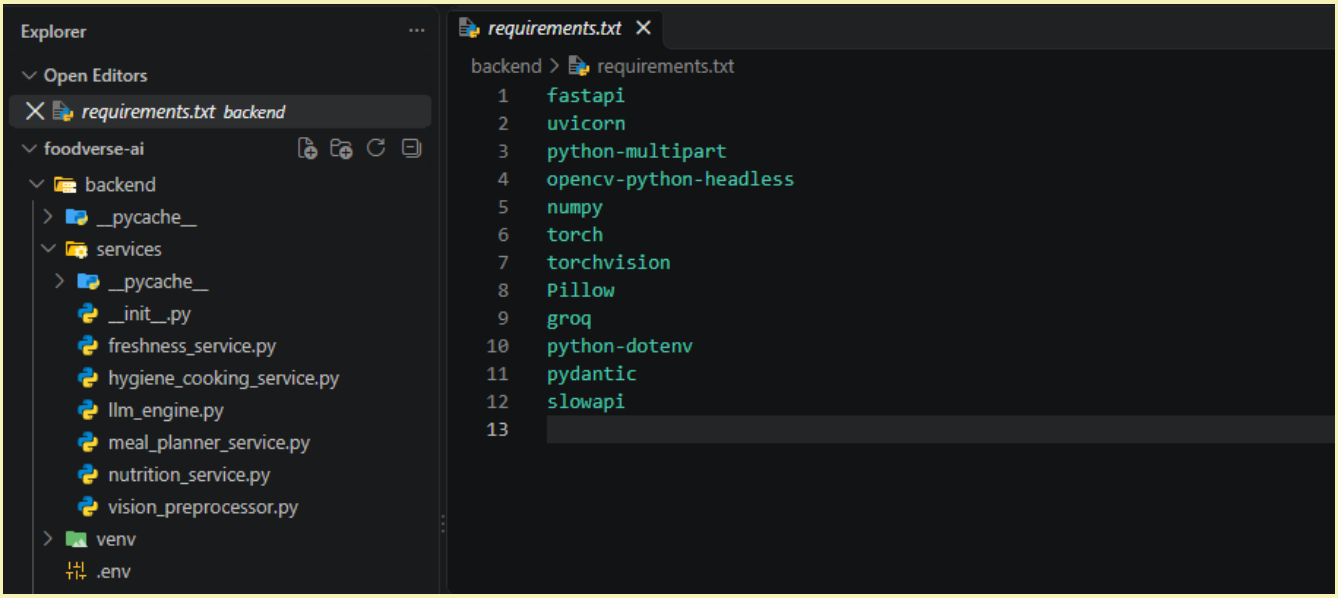
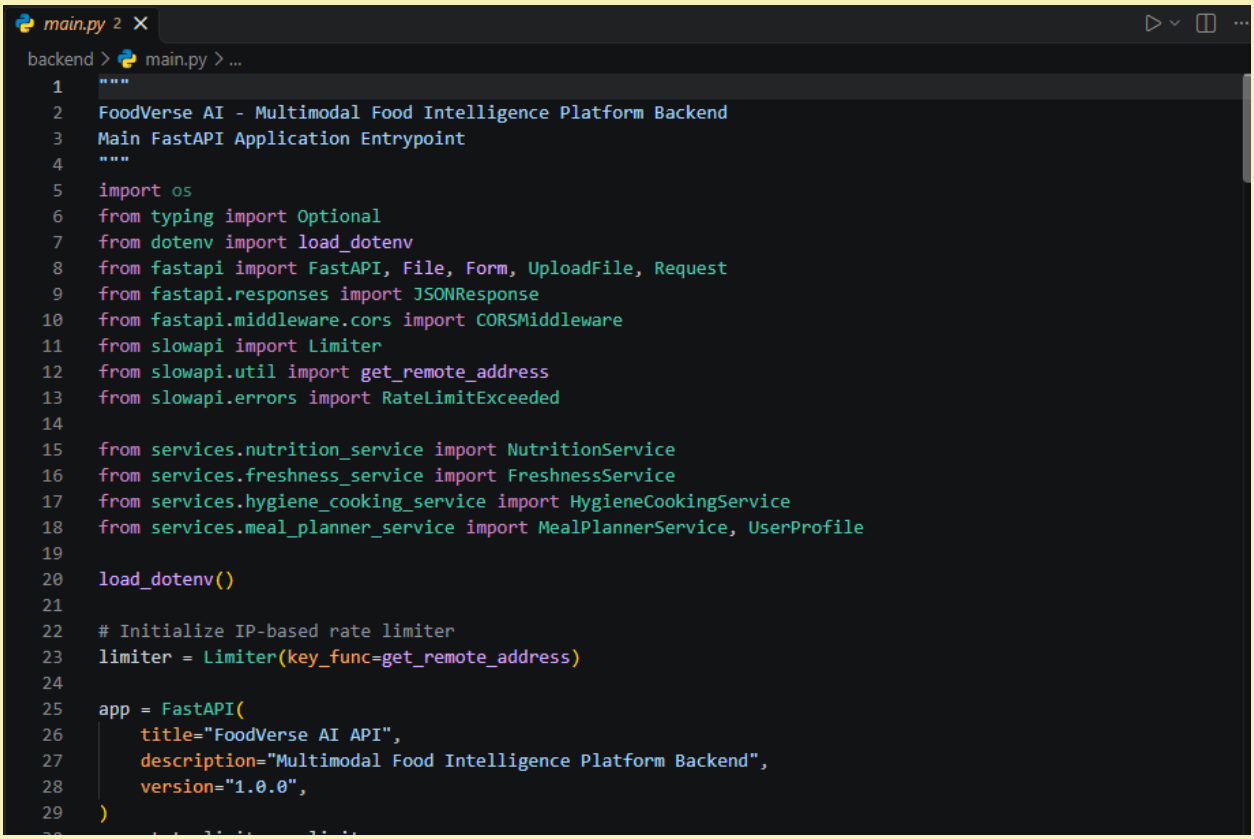
# MILESTONES & ACTIVITIES

## Milestone 1: Requirements Specification & Setup

### Activity 1.1: Create requirements.txt Approach



## Activity 1.2: Install Required Libraries and Set Up Environment



```python
"""
FoodVerse AI - Multimodal Food Intelligence Platform Backend
Main FastAPI Application Entrypoint
"""
import os
from typing import Optional
from dotenv import load_dotenv
from fastapi import FastAPI, File, Form, UploadFile, Request
from fastapi.responses import JSONResponse
from fastapi.middleware.cors import CORSMiddleware
from slowapi import Limiter
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

from services.nutrition_service import NutritionService
from services.freshness_service import FreshnessService
from services.hygiene_cooking_service import HygieneCookingService
from services.meal_planner_service import MealPlannerService, UserProfile

load_dotenv()

# Initialize IP-based rate limiter
limiter = Limiter(key_func=get_remote_address)

app = FastAPI(
    title="FoodVerse AI API",
    description="Multimodal Food Intelligence Platform Backend",
    version="1.0.0",
)
```

### Milestone 2: Food Recognition & Nutrition

## Activity 2.1: Food Image Detection and Classification

nutrition_service.py

backend > services > nutrition_service.py > ...

```python
Activity 2.2 — Nutrition Estimation, Health Scoring & Recommendations

Orchestrates vision_preprocessor → llm_engine → structured nutrition response.
"""
import json
from typing import Any, Dict, List, Optional

from pydantic import BaseModel, Field

from services.vision_preprocessor import VisionPreprocessor
from services.llm_engine import LLMEngine


# ---------------------------------------------------------------------------- #
#  Pydantic response models
# ---------------------------------------------------------------------------- #
class FoodItem(BaseModel):
    name: str
    portion_grams: float = Field(default=100.0, description="Estimated portion weight in grams")
    calories: float = Field(default=0.0, description="Calories in kcal")
    protein_g: float = Field(default=0.0)
    carbs_g: float = Field(default=0.0)
    fats_g: float = Field(default=0.0)
    fiber_g: float = Field(default=0.0)


class NutritionalProfile(BaseModel):
    food_name: str
    portion_grams: float = Field(default=100.0, description="Estimated portion weight in grams")
    calories_kcal: float = Field(default=0.0, description="Calories in kcal")
    protein_g: float = Field(default=0.0, description="Protein in grams")
    carbohydrates_g: float = Field(default=0.0, description="Carbohydrates in grams")
    fats_g: float = Field(default=0.0, description="Fats in grams")
    fiber_g: float = Field(default=0.0, description="Dietary fiber in grams")
```

# Activity 2.2: Nutrition Estimation and Health Scoring Logic

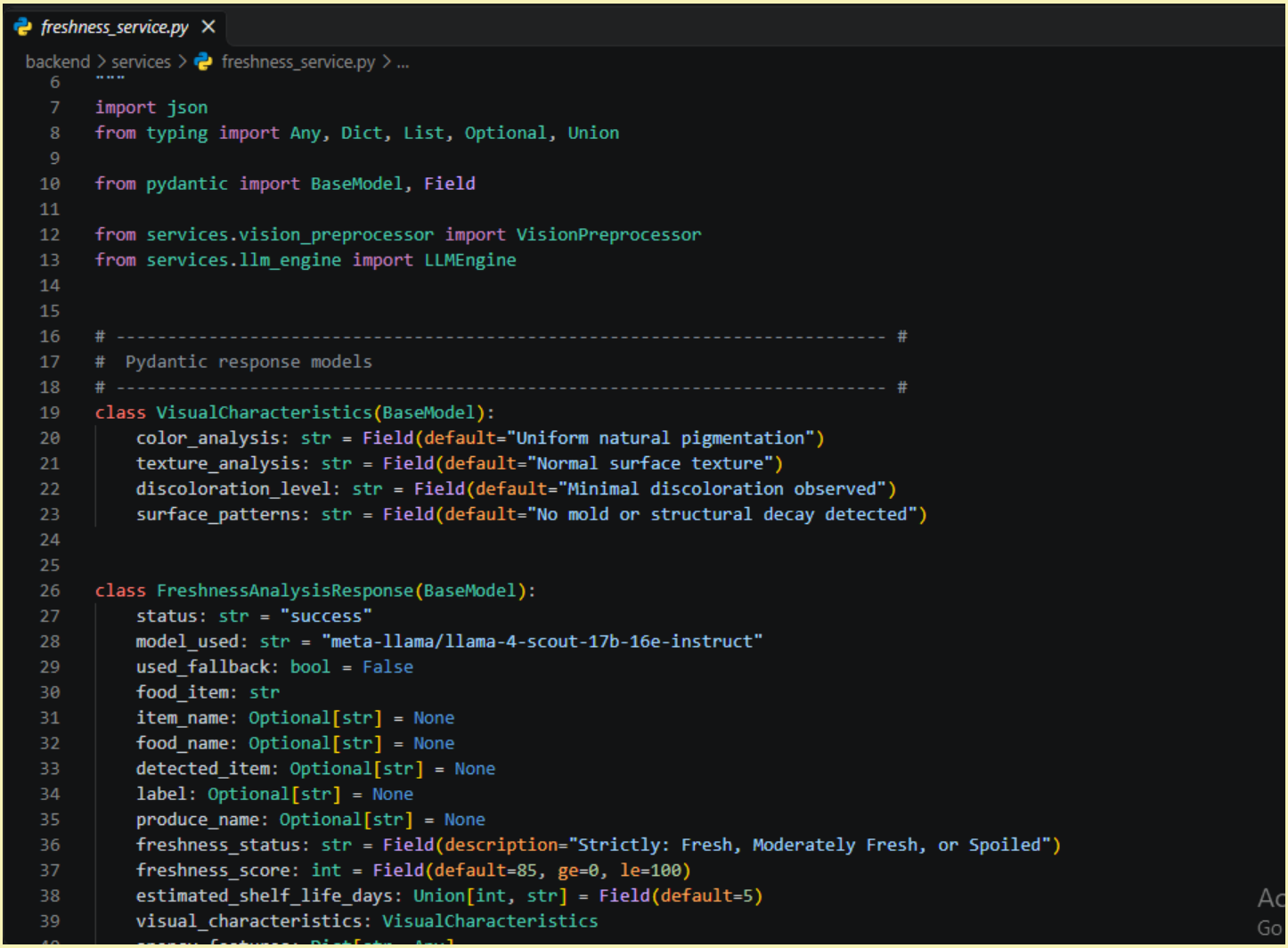
nutrition_service.py

backend > services > nutrition_service.py > ...

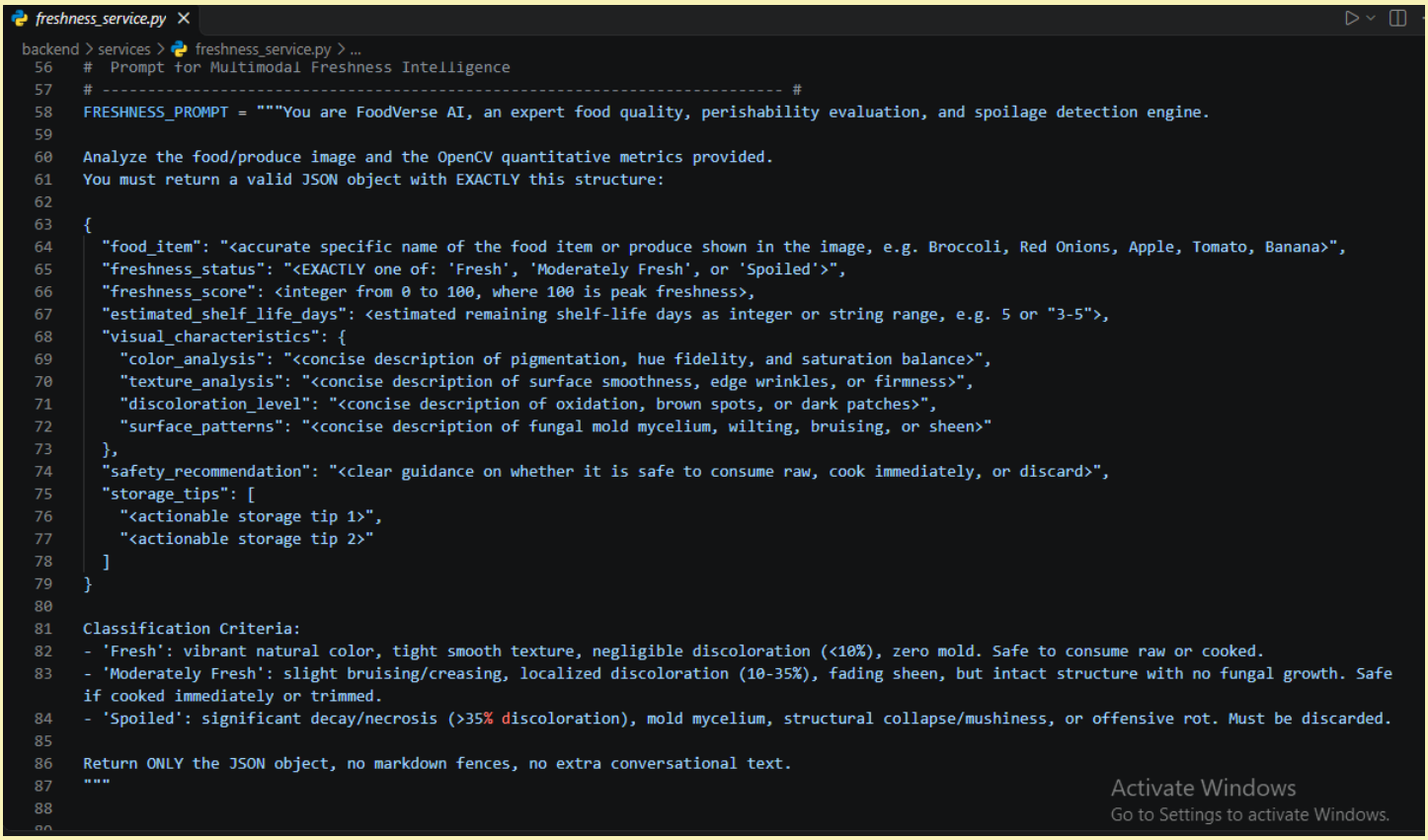
```
54
55  # ------------------------------------------------------------------------ #
56  #  Nutrition analysis prompt
57  # ------------------------------------------------------------------------ #
58  NUTRITION_PROMPT = """You are FoodVerse AI, an expert multimodal food recognition and nutrition analysis engine.
59
60  Analyze the food image provided and return a JSON object with EXACTLY this structure:
61
62  {
63    "food_items": [
64      {
65        "name": "<dish or food item name>",
66        "portion_grams": <estimated portion weight in grams>,
67        "calories": <estimated calories in kcal>,
68        "protein_g": <protein in grams>,
69        "carbs_g": <carbohydrates in grams>,
70        "fats_g": <fats in grams>,
71        "fiber_g": <dietary fiber in grams>
72      }
73    ],
74    "total_calories": <sum of all item calories>,
75    "total_protein_g": <sum>,
76    "total_carbs_g": <sum>,
77    "total_fats_g": <sum>,
78    "total_fiber_g": <sum>,
79    "health_score": <integer 0-100, where 100 is healthiest>,
80    "dietary_tags": ["<e.g. High Protein>", "<Balanced>", "<Low Carb>"],
81    "nutrition_insights": ["<insight 1>", "<insight 2>"],
82    "health_recommendations": ["<recommendation 1>", "<recommendation 2>"]
83  }
84
85  Rules:
86  - Identify EVERY distinct dish / food item visible on the plate or in the image.
87  - Estimate realistic portion sizes and macros based on visual cues.
```

## Milestone 3: Food Freshness & Spoilage Detection

### Activity 3.1: Food Image Preprocessing and Feature Extraction




## Activity 3.2: Freshness Classification & Safety Logic



```
freshness_service.py
backend > services > freshness_service.py > ...
# Prompt for Multimodal Freshness Intelligence
# ------------------------------------------------------------------------ #
FRESHNESS_PROMPT = """You are FoodVerse AI, an expert food quality, perishability evaluation, and spoilage detection engine.

Analyze the food/produce image and the OpenCV quantitative metrics provided.
You must return a valid JSON object with EXACTLY this structure:

{
  "food_item": "<accurate specific name of the food item or produce shown in the image, e.g. Broccoli, Red Onions, Apple, Tomato, Banana>",
  "freshness_status": "<EXACTLY one of: 'Fresh', 'Moderately Fresh', or 'Spoiled'>",
  "freshness_score": <integer from 0 to 100, where 100 is peak freshness>,
  "estimated_shelf_life_days": <estimated remaining shelf-life days as integer or string range, e.g. 5 or "3-5">,
  "visual_characteristics": {
    "color_analysis": "<concise description of pigmentation, hue fidelity, and saturation balance>",
    "texture_analysis": "<concise description of surface smoothness, edge wrinkles, or firmness>",
    "discoloration_level": "<concise description of oxidation, brown spots, or dark patches>",
    "surface_patterns": "<concise description of fungal mold mycelium, wilting, bruising, or sheen>"
  },
  "safety_recommendation": "<clear guidance on whether it is safe to consume raw, cook immediately, or discard>",
  "storage_tips": [
    "<actionable storage tip 1>",
    "<actionable storage tip 2>"
  ]
}

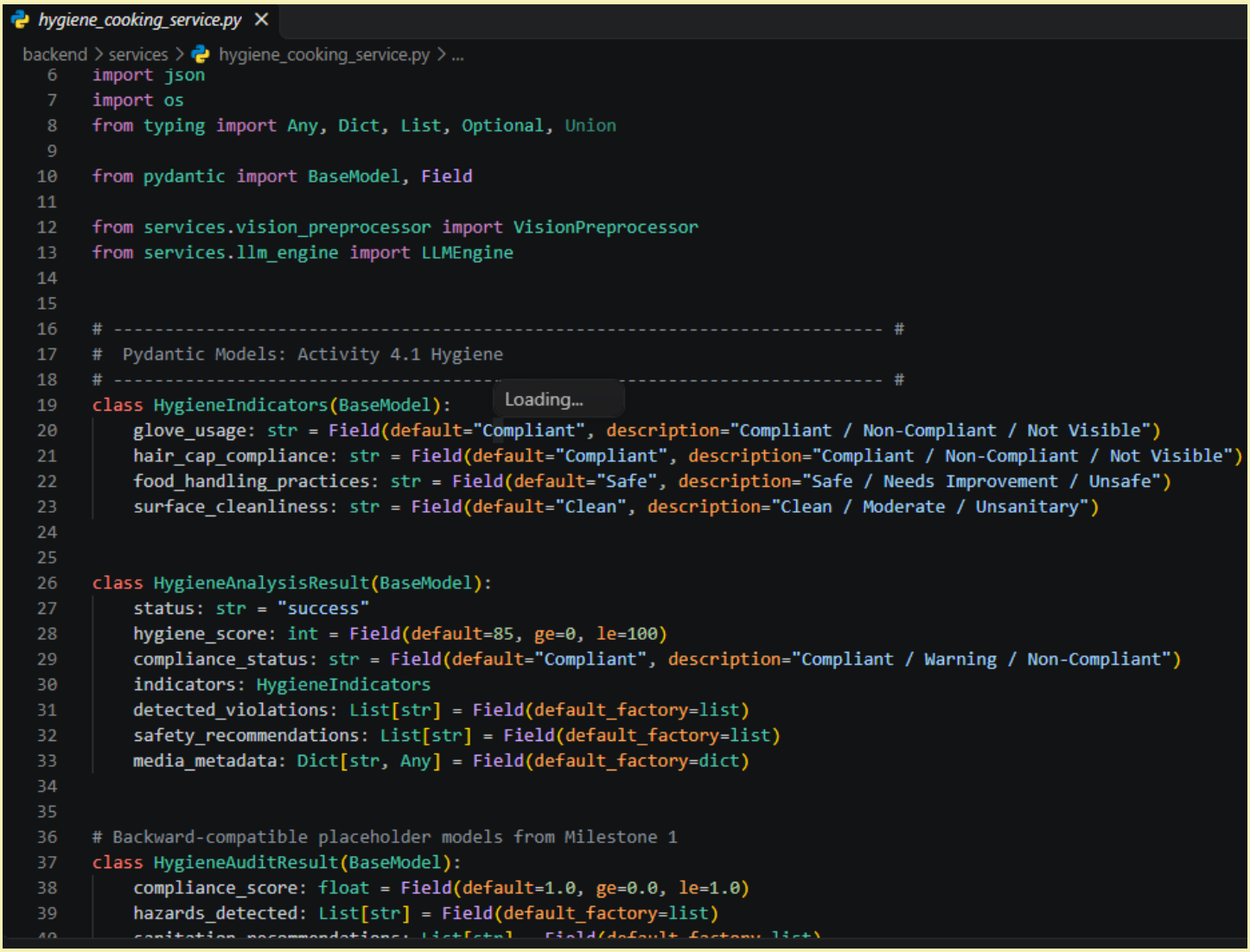
Classification Criteria:
- 'Fresh': vibrant natural color, tight smooth texture, negligible discoloration (<10%), zero mold. Safe to consume raw or cooked.
- 'Moderately Fresh': slight bruising/creasing, localized discoloration (10-35%), fading sheen, but intact structure with no fungal growth. Safe if cooked immediately or trimmed.
- 'Spoiled': significant decay/necrosis (>35% discoloration), mold mycelium, structural collapse/mushiness, or offensive rot. Must be discarded.

Return ONLY the JSON object, no markdown fences, no extra conversational text.
"""
```

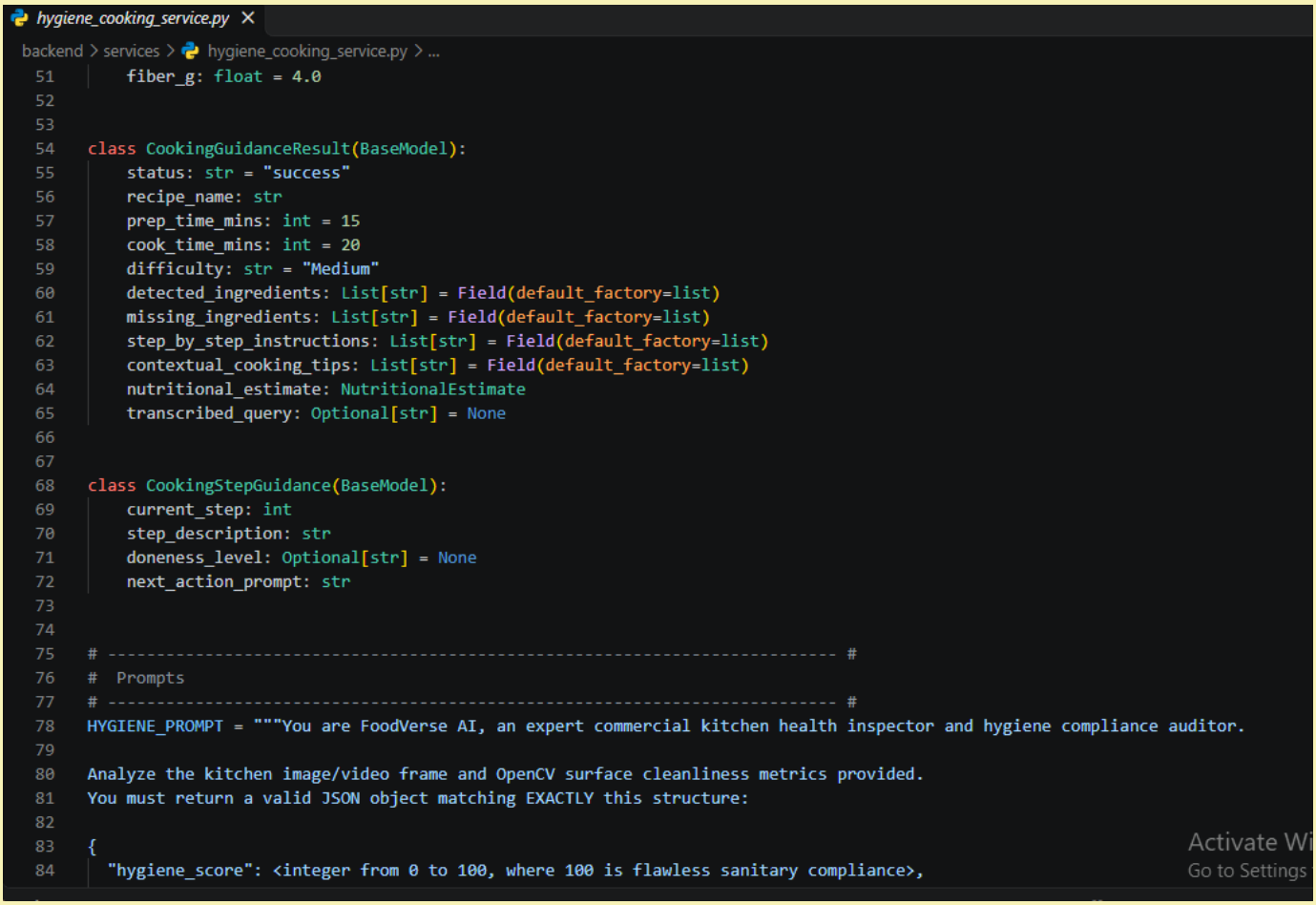
Activate Windows
Go to Settings to activate Windows.

# Milestone 4: Restaurant Hygiene & Smart Kitchen

## Activity 4.1: Restaurant Hygiene Detection and Compliance




## Activity 4.2: Smart Kitchen Cooking Guidance

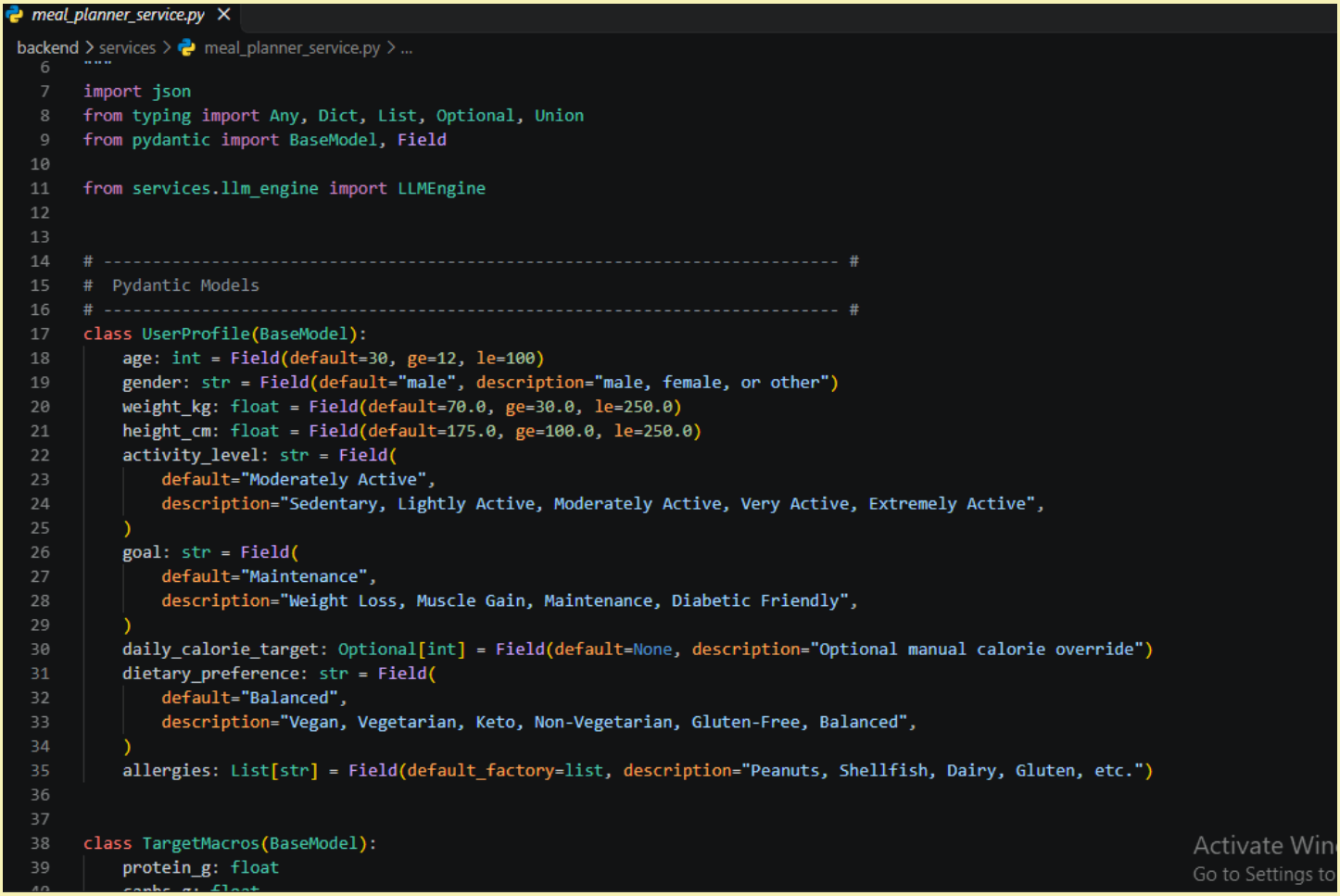


```
hygiene_cooking_service.py
backend > services > hygiene_cooking_service.py > ...
51      fiber_g: float = 4.0
52
53
54  class CookingGuidanceResult(BaseModel):
55      status: str = "success"
56      recipe_name: str
57      prep_time_mins: int = 15
58      cook_time_mins: int = 20
59      difficulty: str = "Medium"
60      detected_ingredients: List[str] = Field(default_factory=list)
61      missing_ingredients: List[str] = Field(default_factory=list)
62      step_by_step_instructions: List[str] = Field(default_factory=list)
63      contextual_cooking_tips: List[str] = Field(default_factory=list)
64      nutritional_estimate: NutritionalEstimate
65      transcribed_query: Optional[str] = None
66
67
68  class CookingStepGuidance(BaseModel):
69      current_step: int
70      step_description: str
71      doneness_level: Optional[str] = None
72      next_action_prompt: str
73
74
75  # ---------------------------------------------------------------------- #
76  #  Prompts
77  # ---------------------------------------------------------------------- #
78  HYGIENE_PROMPT = """You are FoodVerse AI, an expert commercial kitchen health inspector and hygiene compliance auditor.
79
80  Analyze the kitchen image/video frame and OpenCV surface cleanliness metrics provided.
81  You must return a valid JSON object matching EXACTLY this structure:
82
83  {
84    "hygiene_score": <integer from 0 to 100, where 100 is flawless sanitary compliance>,
```

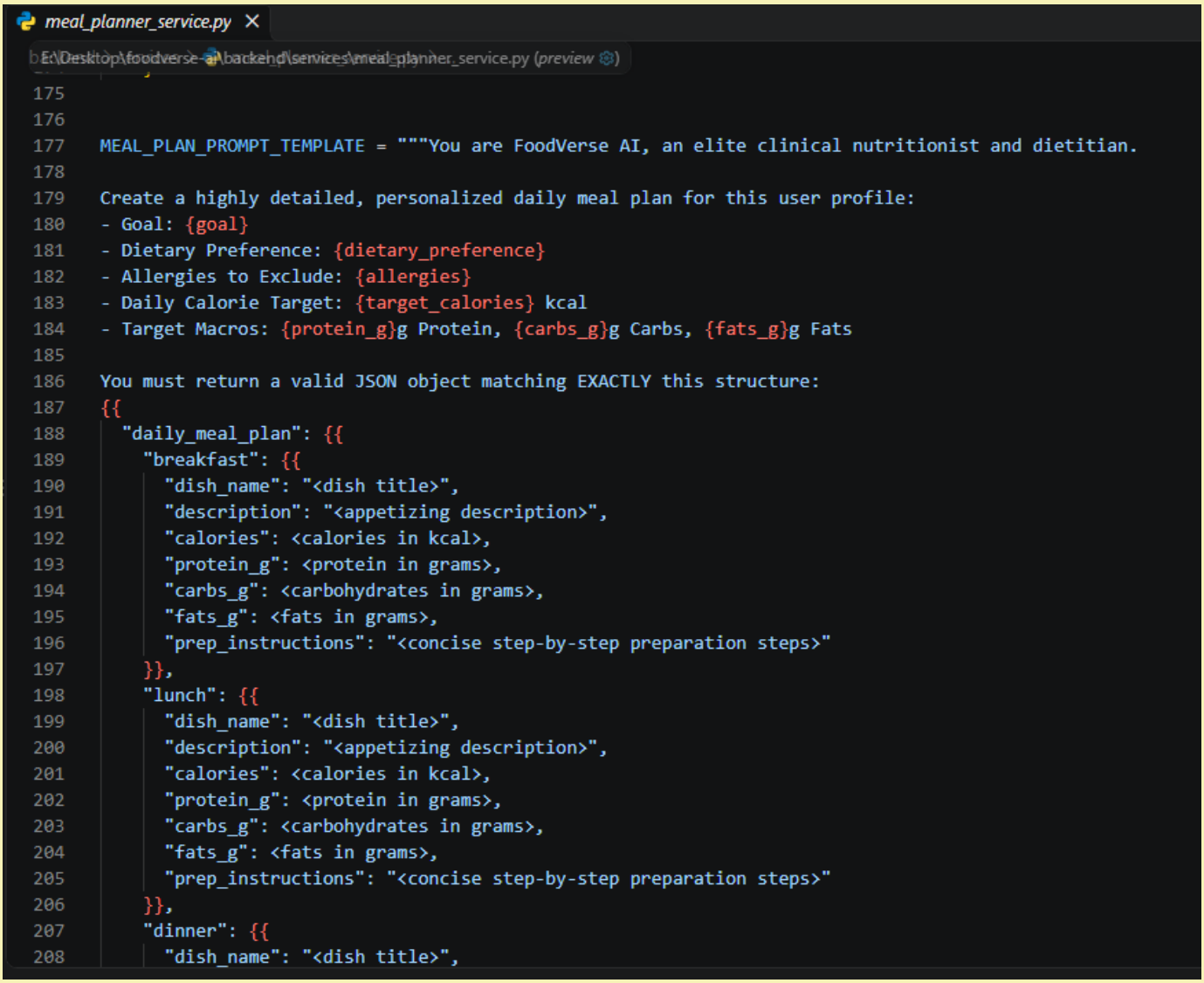
Activate Wi
Go to Settings

## Milestone 5: Personalized Meal Planner Development

Activity 5.1: User Profile, Goal, and Preference Processing




## Activity 5.2: Meal Plan Generation and Recommendations




## Milestone 6: Interface, Testing, and Deployment

### Activity 6.1: Frontend Interface Integration

App.jsx

frontend > src > App.jsx > ...

```jsx
export default function App() {
  const [activeTab, setActiveTabState] = useState('overview');
  const [tabHistory, setTabHistory] = useState([]);

  // Navigate to a new tab while preserving previous history
  const navigateToTab = (nextTab) => {
    if (!nextTab || nextTab === activeTab) return;
    setTabHistory((prev) => {
      if (prev.length > 0 && prev[prev.length - 1] === activeTab) {
        return prev;
      }
      return [...prev, activeTab].slice(-30);
    });
    setActiveTabState(nextTab);
  };

  // Navigate back to the previous tab in history (fallback to overview)
  const handleGoBack = () => {
    setTabHistory((prev) => {
      const newHistory = [...prev];
      let targetTab = 'overview';
      while (newHistory.length > 0) {
        const candidate = newHistory.pop();
        if (candidate && candidate !== activeTab) {
          targetTab = candidate;
          break;
        }
      }
      setActiveTabState(targetTab);
      return newHistory;
    });
  };
```

# Activity 6.2: Functional Testing & Edge Case Validation

test_milestone5.py

backend > test_milestone5.py > ...

```python
def test_tdee_and_macro_calculations():
    print("--- [Activity 5.1] Testing TDEE, BMR & Macro Calculations ---")
    from services.meal_planner_service import UserProfile, calculate_user_summary

    # Male, 28y, 80kg, 180cm, Moderately Active, Goal: Weight Loss
    p_male = UserProfile(
        age=28,
        gender="male",
        weight_kg=80.0,
        height_cm=180.0,
        activity_level="Moderately Active",
        goal="Weight Loss",
        dietary_preference="Balanced",
        allergies=["Peanuts"],
    )
    res_m = calculate_user_summary(p_male)
    print(f"Male Profile: BMR={res_m['bmr_kcal']} kcal, TDEE={res_m['tdee_kcal']} kcal, Target={res_m['target_calories']} kcal")
    print(f"Target Macros: {res_m['target_macros']}")

    # BMR: 10*80 + 6.25*180 - 5*28 + 5 = 800 + 1125 - 140 + 5 = 1790
    assert 1750 <= res_m["bmr_kcal"] <= 1850
    # TDEE: 1790 * 1.55 = 2774
    assert res_m["tdee_kcal"] > res_m["bmr_kcal"]
    # Weight Loss deficit: ~500 kcal
    assert res_m["target_calories"] < res_m["tdee_kcal"]
    assert res_m["target_macros"]["protein_g"] > 0
    assert res_m["target_macros"]["carbs_g"] > 0
    assert res_m["target_macros"]["fats_g"] > 0

    # Female, 30y, 60kg, 165cm, Lightly Active, Goal: Muscle Gain
    p_female = UserProfile(
        age=30,
        gender="female",
        weight_kg=60.0,
```

Activate Windows
Go to Settings to activa

# Activity 6.3: Final Validation and Deployment Readiness

